Python Language Basics

27 July 2024 16:16

Identifiers

In programming languages, identifiers are names used to identify a variables, functions or other entities in a program. The rules for naming an identifier in Python are as follows:

- The name should begin with an uppercase or a lowercase alphabet, or an underscore '_' sign. This may be followed by any combination of characters A-Z, a-z , 0-9 or underscore'_'. Thus, an identifier cannot start with a digit.
- It can be of any length.(However, it is preferred to keep it short and meaningful.)
- It should not be a keyword or resolved word of Python.
- We cannot use. symbols like: .,!,?,@,$,#,%,&,*,() etc. , in identifiers.

For example:

```
A = 10
B = 20
Name = "Jiya"
 area = length * breadth
```

Python Keywords

Keywords are reserved words. each keyword has a specific meaning to the python interpreter. and we can use a keyword in our program only for the purpose for which it has been defined. as Python is case sensitive, keyword must be written exactly.

```python
import keyword
print("The list of keywords")
kw = keyword.kwlist
# print(kw)
slno = 1
for k in kw:
    print(slno,'. ',k, sep='')
    slno += 1
```

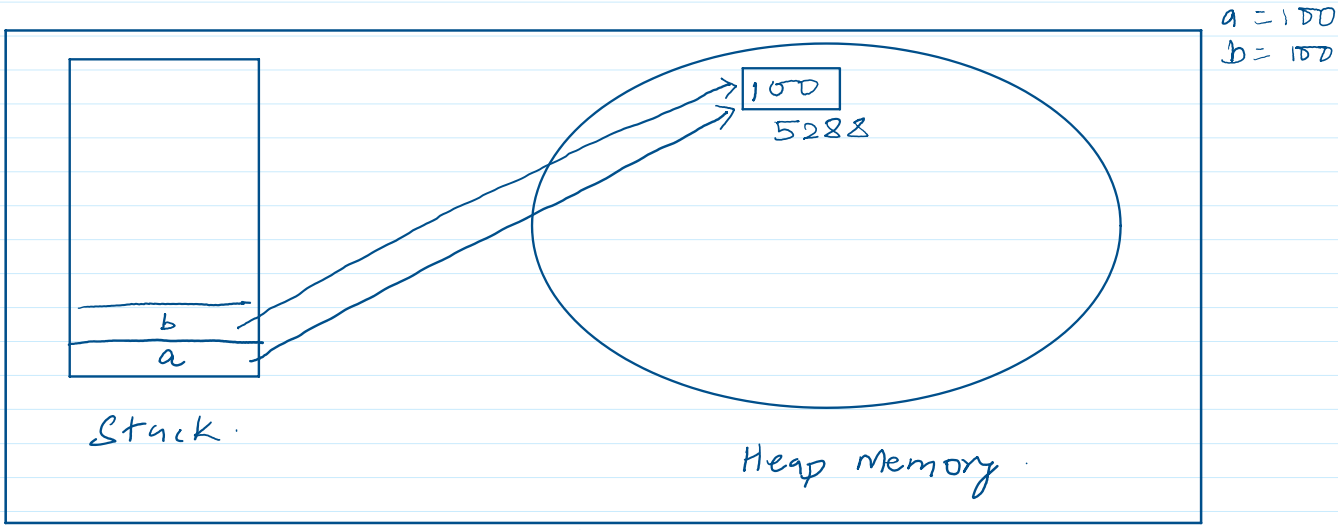
Output:

```
The list of keywords
1. False
2. None
3. True
4. and
5. as
6. assert
7. async
8. await
9. break
10. class
11. continue
12. def
13. del
14. elif
15. else
16. except
17. finally
18. for
19. from
20. global
21. if
22. import
23. in
24. is
25. lambda
26. nonlocal
```

27. not
28. or
29. pass
30. raise
31. return
32. try
33. while
34. with
35. yield

Variables

A variable in a program is uniquely identified by a name (identifier) variable in Python referred to an object. And item or element that is stored in the memory.




```
>>> a = 100
>>> b = 100
>>> id(a)
140725993035288
>>> id(b)
140725993035288
```

RAM
[Random Access Memory].

Types of Memory
- → Primary memory (volatile) → RAM
- → Secondary memory (permanent) → SSD, [HDD]

SSD
Solid State Drive.
[Silicon Technology]
[Faster]

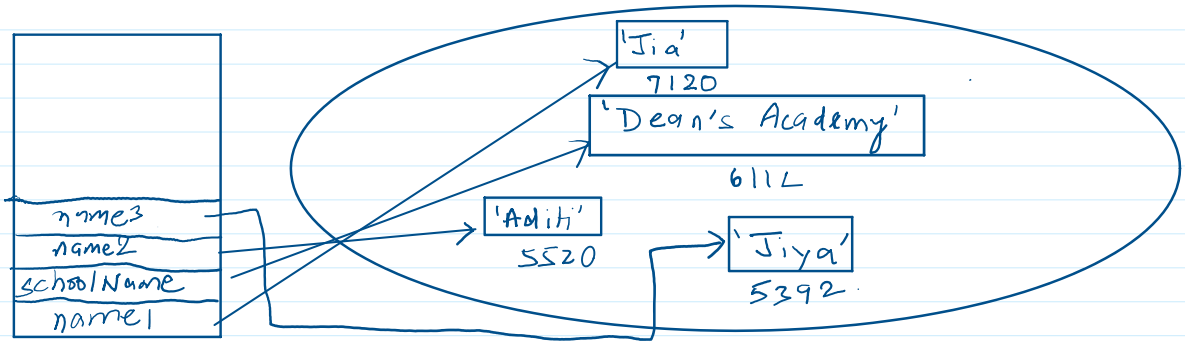
HDD
Hard Disk Drive.
[Magnetic Technology]
[Slow]

```
>>> name1 = "Jia"
>>> schoolName = "Deen's Academy"
>>> name2 = "Aditi"
>>> name3 = "Jiya"
>>> print(name1,'is studying in', schoolName)
Jia is studying in Deen's Academy
>>> print(name2,'is studying in', schoolName)
Aditi is studying in Deen's Academy
>>> print(name3,'is studying in', schoolName)
```

```
>>> id(name1)
2503093067120
>>> id(schoolName)
2503094416112
>>> id(name2)
2503092845520
>>> id(name3)
2503092835392
```

```
>>> print(name2,'is studying in', schoolName)
Aditi is studying in Deen's Academy
>>> print(name3,'is studying in', schoolName)
Jiya is studying in Deen's Academy
>>>
```

```
2503092845520
>>> id(name3)
2503092835392
```



Comments

Comments are used to add a remark or a note in the source code. Comments are not executed by interpreter. They are added with a purpose of making the source code easier for humans to understand. They are used primarily to document the meaning and purpose of source code. And its. input and output requirements so that we can remember later how its functions and how to use it. for large and complex software, it may require programmers to work in team and sometimes a program is written by one programmer is required to be used or maintained by another programmer. In such situations, documentation in the form of comments are needed to understand the working of the program.

In Python, a comment start with hash sign '#'. Everything following the hash till the end of that line is treated as comment and the interpreter simply ignores it while executing the statement.

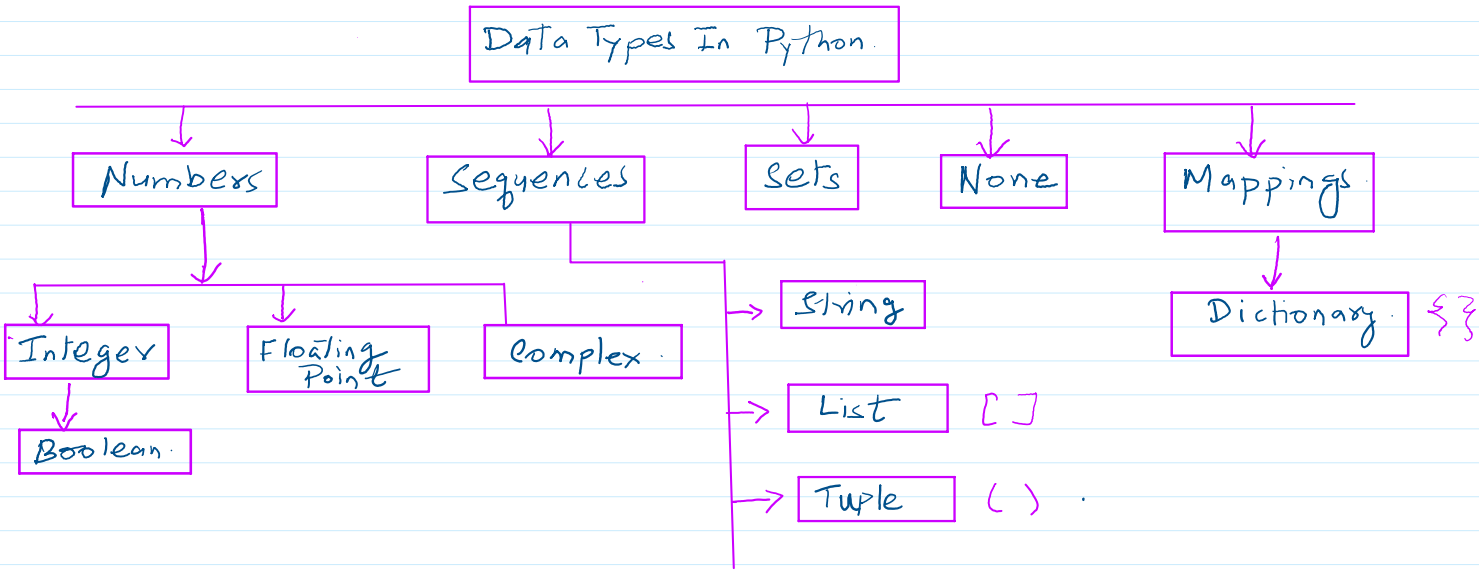
"Comments"

```
# to find the sum of two integers
x = int(input("enter any integer value: "))
y = int(input("enter any integer value: "))
sum = x + y
print("Sum of two integers: ", sum)
```

```
# Variable amount is the total spending
# on grocery
amount = 3400
# totalMarks is the sum of all the test
# of mathematics
test1 = test2 = test3 = test4 = test5 = test6 = test7 = 15
totalMarks = test1 + test2 + test3 + test4 + test5 + test6 + test7
print("Total marks: ", totalMarks)
```

Data Types

Every value belongs to a specific data type in python. Data type identifies the type of data values of variable can hold, and the operation that can be performed on that data types.



Different Data types in Python

$4^2 = 16$

$4^{-2} = \frac{1}{16}$

$4^{1/2} =$ or

$\sqrt{4} = \sqrt{2 \times 2} = 2$

$\sqrt{9} = \sqrt{3 \times 3} = 3$

$\sqrt{8} = 2.\ldots$

Number

Number data type stores numerical values only. It is further classified into three different types: int, float, and complex.

Type/Class	Description	Examples
int	Integer numbers.	**-12, -3, 0, 125, 2**
float	Real or floating point numbers.	**-2.04, 4.0, 14.66**
complex	Complex numbers	**3 + 4j, 2 - 2j**
bool	Boolean values	**True, False**

Sqrt(8) = 2.82842712474619

Sqrt(-8) =

$i = \text{"}\frac{9}{H}\text{"}$

j

$\sqrt{-8} = \sqrt{(-1) \times 2 \times 2 \times 2}$

$i = (\sqrt{-1})$

iota

$= 2 \times \sqrt{(-1)} \times \sqrt{2}$

$= \boxed{2\sqrt{2}\, i}$ ← Math

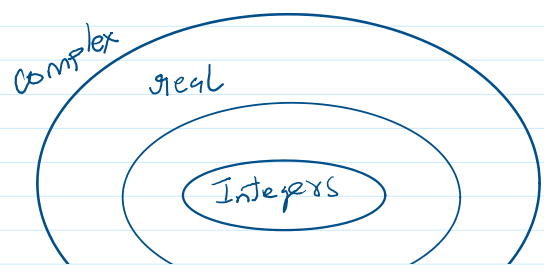
$(2\sqrt{2}\, i)$ ✗ ← Python

$(2\sqrt{2}\, j)$ ✓

```
>>> x = 10
>>> type(x)
<class 'int'>
>>> y = 10.24
>>> type(y)
<class 'float'>
>>> c1 = 0j
>>> type(c1)
<class 'complex'>

>>> import math as M
>>> c2 = 2 * M.sqrt(2)
>>> c2
2.8284271247461903

>>> c3 = complex(c2)
>>> c3
(2.8284271247461903+0j)
```

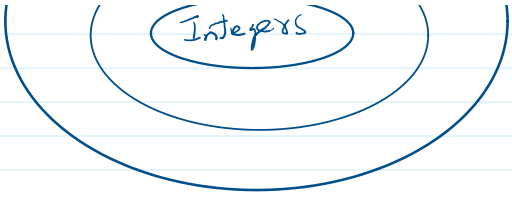


```
>>> c3 = complex(c2)
>>> c3
(2.8284271247461903+0j)

>>> c4 = 4 + j
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'j' is not defined
>>> c4 = 4 + 0j
>>> c5 = 5 + 16j
>>> type(c5)
<class 'complex'>
>>>
```

← <any numerical_value>j

← correct data type



More examples:

```
isLetter = True
isLetter
True
isLetter = False
isLetter
False
type(isLetter)
<class 'bool'>
x = 10
x
10
type(x)
<class 'int'>
y = 5.67
type(int)
<class 'type'>
type(y)
<class 'float'>
type(float)
<class 'type'>
c = 0j
type(c)
<class 'complex'>
c1 = 5 + 6j
c1
(5+6j)
type(c1)
<class 'complex'>
```

Sequences

A Python sequence is an ordered collection of items where each item is indexed by an integer. The three types of sequence data types, available in Python are string, list, and tuples.

String

String is a group of characters. These characters may be alphabets digits or special characters, including spaces. String values are enclosed either in single quotation mark. or in double quotation mark. The quotes are not the part of string. They are used to mark the beginning and the end of the string for the interpreter. For example:

```
>>> str1 = 'Hello, Friends'
>>> type(str1)
<class 'str'>
>>> str1
```

```
>>> type(str1)
<class 'str'>
>>> str1
'Hello, Friends'
>>> print(str1)
Hello, Friends
>>>
```

← internally Python puts single quotes for the string.
← if we use print() method then quotes are not printed.

```
>>> str2 = "Shiv"
>>> str2
'Shiv'
>>>
```

[] ← Bracket
{} ← Brace.
() ← Parenthesis.

List
List is a sequence of items separated by commas And the items are enclosed in square brackets [].

For example:

```
>>> fruits = ['Mango', 'Apple', 'Banana', 'Litchi']
>>> type(fruits)
<class 'list'>
>>> fruits
['Mango', 'Apple', 'Banana', 'Litchi']
>>> print(fruits)
['Mango', 'Apple', 'Banana', 'Litchi']
```

Tuple
Tuple is a sequence of items separated by commas and items are enclosed in parentheses (). This is unlike list where values are enclosed in brackets[]. Once created, we cannot change the tuple. For example:

```
>>> vegetables = ('Potatoes', 'Tomatoes', 'Carrot', 'Cucumber', 'bitter gourd')
>>> type(vegetables)
<class 'tuple'>
>>> vegetables
('Potatoes', 'Tomatoes', 'Carrot', 'Cucumber', 'bitter gourd')
```

Sets
Set is an unordered collection of items separated by commas. And the items are enclosed in braces {}. A set is similar to list, except that it cannot have duplicate entries. Once created, elements of a set cannot be changed.
For example:

```
>>> vegetableSet = {'Potatoes', 'Tomatoes', 'Carrot', 'Cucumber', 'bitter gourd'}
>>> type(vegetableSet)
<class 'set'>
>>> num = {10,20,30,40,50}
>>> type(num)
<class 'set'>
>>> num
{50, 20, 40, 10, 30}
>>> vegetableSet
{'bitter gourd', 'Potatoes', 'Tomatoes', 'Carrot', 'Cucumber'}
```

None
None is a special data type with a single value. It is used to signify the absence of a value in situation. None supports no special operation. And it is neither False nor 0(zero).
For example:

```
>>> myVar = None
>>> type(myVar)
<class 'NoneType'>
>>> print(myVar)
None
```

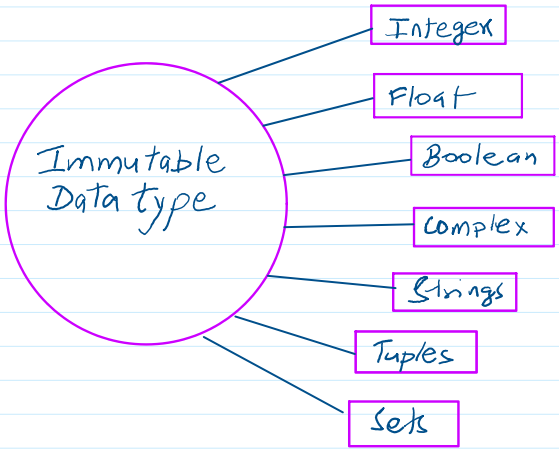
Mapping
Mapping is an unordered data type in Python. Currently, there is only one standard mapping data type in Python called dictionary.

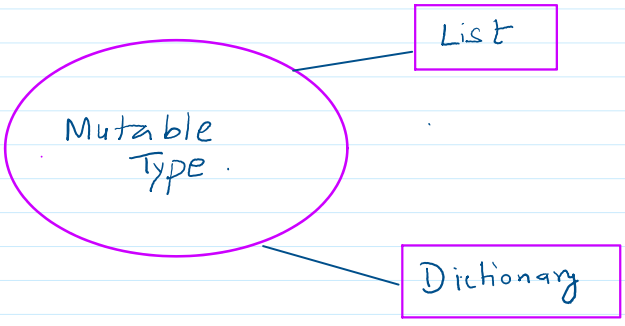
Dictionary
Dictionary in python holds data item in key-value pairs. Items in a dictionary are enclosed in braces {} Dictionaries permits faster access to data. Every key is separated from a value using a colon(:) sign. The key:value pair of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type. In order to access any value in dictionary, we have to specify key in brackets [].

```
>>> dict1 = { 'Fruit': 'Mangoes', 'Climate': 'Hot and humid', 'Price(Kg)': 200 }
>>> dict1['Climate']
'Hot and humid'
>>> dict1['Price(Kg)']
200
```

Mutable and Immutable Data types

Variables whose value can change after they are created and assigned are called **mutable.**
variables whose value cannot be change. after they are created and assigned are called **immutable.**
When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in memory. Therefore it consumes time and processing power.





Home Work

Exercise

1. Which of the following identifier names are invalid and why?

i	`Serial_no.`	v	`Total_Marks`
ii	`1st_Room`	vi	`total-Marks`
iii	`Hundred$`	vii	`_Percentage`
iv	`Total Marks`	viii	`True`

2. Write the corresponding Python assignment statements:
   a) Assign 10 to variable `length` and 20 to variable `breadth`.
   b) Assign the average of values of variables `length` and `breadth` to a variable `sum`.
   c) Assign a list containing strings 'Paper', 'Gel Pen', and 'Eraser' to a variable `stationery`.
   d) Assign the strings 'Mohandas', 'Karamchand', and 'Gandhi' to variables `first`, `middle` and `last`.
   e) Assign the concatenated value of string variables `first`, `middle` and `last` to variable `fullname`. Make sure to incorporate blank spaces appropriately between different parts of names.
3. Write logical expressions corresponding to the following statements in Python and evaluate the expressions (assuming variables `num1`, `num2`, `num3`, `first`, `middle`, `last` are already having meaningful values):
   a) The sum of 20 and –10 is less than 12.
   b) `num3` is not more than 24.
   c) 6.75 is between the values of integers num1 and `num2`.
   d) The string 'middle' is larger than the string 'first' and smaller than the string 'last'.
   e) List `Stationery` is empty.

© NCERT not to be republished